AI-POWERED NO-CODE APPLICATION GENERATOR

with Automated Validation and Testing

WORKING PROTOTYPE

Foundations of Artificial Intelligence

Course Code: CS23533

SRI AYYAPPA S • 240701526

Faculty Guide: MUNEESHWARI P

RAJALAKSHMI ENGINEERING COLLEGE

Abstract

From plain language to a verified working application

The proposed system converts a natural-language description of a role-based CRUD application into an intermediate structured specification and then generates a complete working prototype. The pipeline validates the specification at both structural and logical levels before generating SQL, FastAPI backend services, and HTML/JavaScript frontend pages. It also derives role–action–entity test cases and executes them against the generated application. The prototype has been verified on two application types—attendance management and library management—using the same generator pipeline.

Input

Plain-English application description

Reasoning layer

Structured spec + dual validation

Output

Database + backend + frontend + tests

Problem Statement

The development gap

Turning a natural-language idea into a usable application still requires multiple technical decisions: data model, permissions, APIs, UI, and testing.

AI code-generation systems can produce code quickly, but generated code may contain structural or logical inconsistencies.

For role-based CRUD applications, incorrect permissions or mismatched entities can produce applications that appear functional but violate the intended specification.

A reliable no-code generator therefore needs an explicit representation of requirements and automated verification—not generation alone.

Requirement ambiguity

Natural language → executable requirements

Generation errors

Schema, permissions, APIs or UI can disagree

Verification gap

A generated app needs systematic tests

Literature Survey

Codex / HumanEval (Chen et al., 2021)

Demonstrated that LLMs can synthesize executable code from natural-language descriptions, while also exposing functional-correctness limitations.

NL2Code Survey (Zan et al., 2022)

Surveyed 27 code-oriented LLMs and highlighted NL2Code as a major challenge with gaps between generated programs and human requirements.

LLM Automated Program Repair (Fan et al., 2022)

Showed that generated code can contain common programming mistakes and that repair techniques can improve reliability.

Replit Agent / AI builders

Modern systems can generate full applications from prompts and support iterative refinement; Replit also describes automated testing and fixing loops.

v0 / Vercel

Uses natural-language prompting to create working applications and supports planning, databases, APIs, iteration and deployment.

Lovable

Generates interfaces, data, logic and authentication from a prompt, with conversational refinement and deployment.

Sources: Chen et al. (2021); Zan et al. (2022); Fan et al. (2022); Replit; Vercel v0; Lovable

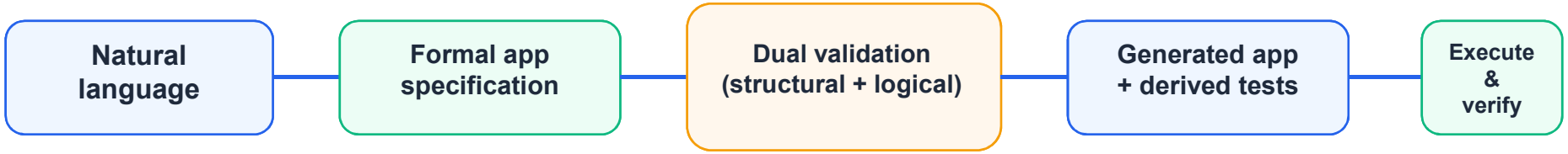
Existing Systems & Limitations

System	Natural-language generation	Iteration / testing	Our research focus
Replit Agent	Yes	Iterative refinement; automated testing/fixing described	Broad, open-ended app creation
v0	Yes	Conversational iteration + design mode	Strong web/UI generation
Lovable	Yes	Chat-based refinement + security scanning	Full-stack app creation
Our prototype	**Yes***	**Spec-derived tests executed on generated app**	**Formal intermediate spec + dual validation + systematic CRUD test derivation**

* Live Gemini call is designed but currently unavailable; prototype uses a documented mock/stub extraction layer.

Research Gap

Gap identified from the reviewed literature and current AI app builders



Research gap

A need for a transparent intermediate representation that can be validated before code generation.

Reliability gap

Generation should be coupled to systematic, specification-derived verification.

Scope gap

A constrained CRUD domain allows the pipeline to be evaluated end-to-end instead of claiming arbitrary app generation.

This prototype addresses the gap through an explicit specification → validation → generation → testing pipeline.

Research Objectives

01 Translate

Convert a plain-English role-based CRUD application description into a structured specification.

02 Validate

Check the specification using JSON Schema and a custom logical/CSP validator.

03 Generate

Produce a real SQLite schema, FastAPI backend, and HTML/JavaScript frontend from the validated specification.

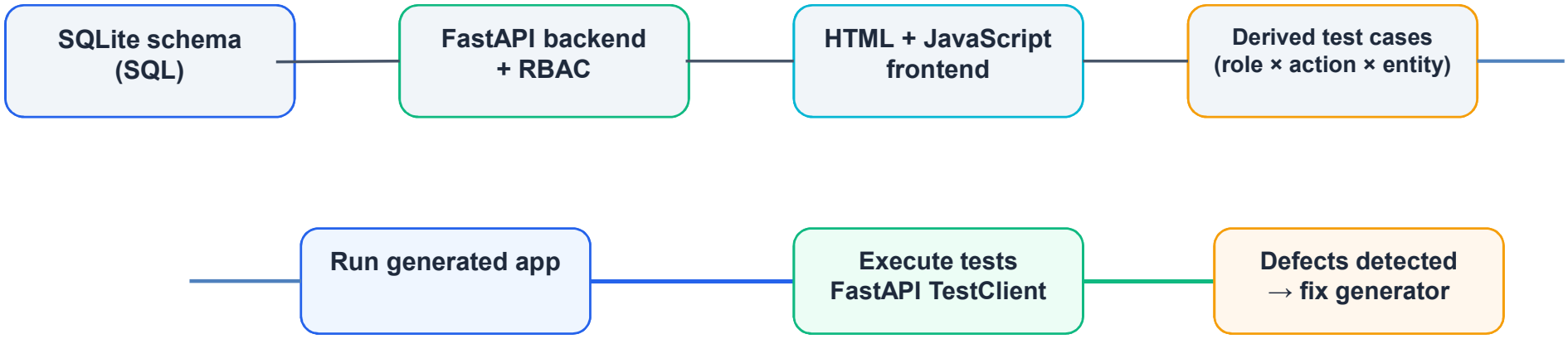
04 Verify

Automatically derive and execute test cases across roles, actions and entities to detect defects in the generated application.

Proposed Work — System Architecture



Generation outputs



Algorithm / Workflow

1. **Capture prompt** — Receive the user's natural-language description.
2. **Extract spec** — Convert it into roles, permissions, entities, fields and pages.
3. **Validate** — Run JSON Schema validation, then logical/CSP consistency checks.
4. **Generate** — Create SQL, FastAPI routes/permission checks, and frontend pages.
5. **Derive tests** — Generate role × action × entity test cases from the same specification.
6. **Execute** — Launch the generated application and run HTTP-style tests.
7. **Report** — Record failures and use them to correct the generator.

Key property

Tests are derived from the same specification that drives generation.

Implementation & Technologies

Language
Python

AI / LLM
Gemini API (designed); mock/stub currently used

Backend target
FastAPI

Schema validation
jsonschema

Database
SQLite

Logical validation
Custom CSP-based validator

Frontend target
HTML + Vanilla JavaScript

Testing
FastAPI TestClient

Design decision: process-level sandboxing is used for the current prototype; full Docker sandboxing was intentionally kept outside the present scope.

Output & Results

Verified prototype results

2 application domains

Attendance management and library management were generated using the same generator code.

Real artifacts

The pipeline produces executable SQL, FastAPI backend code, and HTML/JavaScript frontend pages.

Role-based controls

Backend permission checks are mechanically derived from the application specification.

Automated verification

Generated test cases are executed against the generated application using FastAPI TestClient.

2 genuine defects found & fixed

Testing exposed a database connection leak and a test-data collision; both were corrected in the generator.

Evidence available for demonstration

- Terminal test runs with passing results
- Generated project folders
- Generated SQL schema
- FastAPI routes and permission checks
- Browser run of generated login/dashboard pages can be captured for the final presentation

Current evidence is code/test based; no synthetic screenshots are presented as if they were real.

Discussion: What Is Actually Implemented?

Implemented & verified

✓ Plain-English input
✓ Structured JSON application specification
✓ JSON Schema validation
✓ Custom CSP/logical validation
✓ SQL generation
✓ FastAPI backend generation
✓ Role-based permission checks
✓ HTML/JavaScript frontend generation
✓ Automatic test-case derivation
✓ Live execution against generated apps
✓ Verification on attendance + library domains

Not yet implemented

○ Follow-up prompt modification of an existing app
○ Full Docker-based sandboxing
○ Mobile/APK generation
○ Arbitrary-domain app generation
○ Live Gemini extraction in the current environment

The current scope is intentionally limited to role-based CRUD applications.

Limitations & Future Scope

Current limitations

- Gemini API access is currently blocked by a free-tier 429 RESOURCE_EXHAUSTED / limit: 0 response; mock mode is used.
- Generator scope is limited to role-based CRUD applications.
- Existing-app modification is not implemented yet.
- Current sandbox is process-level rather than fully containerized.
- No mobile/APK generation in the present scope.

Planned extensions

1. Activate and evaluate live Gemini extraction.
2. Add specification-aware follow-up modification.
3. Add stronger isolation with Docker/containers.
4. Expand domain templates and supported application patterns.
5. Add richer UI frameworks and deployment targets after reliability is established.
6. Evaluate generation success, defect rate and test coverage quantitatively.

Conclusion

A generator should not stop at code generation.

Our prototype demonstrates an end-to-end, specification-driven pipeline for constrained no-code application generation.

1 Structure

Natural language is converted into an explicit application specification.

2 Validate

Structural and logical consistency are checked before generation.

3 Verify

Tests derived from the specification are executed against the generated app.

Result: a working prototype validated across two different role-based CRUD application types.

References

1. Chen, M. et al. (2021). Evaluating Large Language Models Trained on Code. arXiv:2107.03374.

2. Zan, D. et al. (2022). Large Language Models Meet NL2Code: A Survey. arXiv:2212.09420.

3. Fan, Z. et al. (2022). Automated Repair of Programs from Large Language Models. arXiv:2205.10583.

4. Zhang, Q. et al. (2023). A Critical Review of Large Language Model on Software Engineering: An Example from ChatGPT and Automated Program Repair. arXiv:2310.08879.

5. Replit. Replit Agent / AI App Builder documentation and product pages. Accessed Sep. 2026.

6. Vercel. v0 — AI-powered full-stack web application builder and documentation. Accessed Sep. 2026.

7. Lovable. AI App Builder — natural-language application generation, data, logic and deployment. Accessed Sep. 2026.

Web sources used for the literature survey and current product comparison:

replit.com/build • v0.dev • lovable.dev/build/app-builder